

Dépouillement REAC 37827 "développeur IA" - compétences

cf. pour les remarques générales :

<https://docs.google.com/document/d/1UwhXSs8h8nO-uFmDzmNnbBqQ5ewJaYl49PXXKogjSgEc/edit?tab=t.0>

Note : ce document propose une interprétation du REAC, il ne s'y substitue pas.

E1 : collecter, stocker et mettre à disposition des données

(Tout le bloc 1 est dans cet ensemble de compétences)

Le but est de récupérer des données de 5 formats différents via du code, de les cleaner, les harmoniser en DB, les agréger, et les exposer via une API sécurisée.

- Le temps sera limité sur E1 : vous présentez ici le projet + E1 lui-même, en 15 min. Ne prenez pas trop de temps sur l'intro, allez à l'essentiel
- Vous devez prouver au jury que vous savez : scraper un site, parser du HTML, fusionner des données, et créer une API. Bref, **démontrer votre capacité à interagir avec les cinq types de sources**
- On vous conseille fortement d'avoir un projet qui tourne pour E1, éventuellement avec une petite démo (attention quand même au temps !)
- Vous pouvez même mettre le lien du repo dans le rapport pour que le jury puisse y accéder en amont de la soutenance
- Le jury pourra poser des questions complémentaires : vous devez être capable d'expliquer le processus de traitement, les difficultés rencontrées, etc.
- Technos recommandées : Python, requests/BeautifulSoup (pour le scrapping), données en CSV, FastAPI, Postgres, sqlalchemy
- Identifiez un thème de données, puis des sources pour vos 5 sources (ex: le cinéma, des données sur l'eau...)
- Une fois les données collectées, faites l'agrégation d'au moins deux sources en SQL : soit via un JOIN, soit en concaténant deux tables
- Le choix des sources n'a aucune importance, mais il faut que les données aient la même nature : si vous agrégez des ventes de voiture, les 5 sources doivent concerner des ventes de voitures. Critère : **toutes les données doivent pouvoir arriver dans une même table à l'arrivée.**
- Rapport : pas besoin de placer l'intégralité des scripts ni le rapport ou les annexe, seulement les parties particulièrement pertinentes pour démontrer les compétences

C1 : automatiser l'extraction des données

Vous devez automatiser la récupération de données issues de 5 sources différentes :

1. une Web API
2. du Web scraping
3. un fichier (CSV, JSON...)
4. une DB (SQL ou NoSQL)
5. un système big data (DuckDB, Spark, Hadoop, Cassandra...) à voir si MongoDB passe

- Identifier le thème commun de vos données, puis des sources, et en faire l'agrégation
- La certif est agnostique de techno, mais il faut pouvoir justifier globalement **en quoi les données demandent un traitement particulier** (volume ? non-structurée ?) et en quoi la techno que vous avez choisie est appropriée pour ce traitement
- Un critère de validation est que "le script est fonctionnel" (même si ça ne sera pas vérifié car E1 n'est pas une démo)
- La "destination" évoquée est une DB, idéalement relationnelle (sauf si vous prouvez le besoin de NoSQL)

Conseil : proposez un tableau récapitulatif de vos 5 sources :

- la source (API, scraping, fichier, DB, big data)
 - la provenance (Kaggle, data.gouv.fr, API interne de votre entreprise...)
 - le type de données (vente de grille-pains en Bulgarie en 2013, playlists...)
 - le format (HTML, CSV, JSON...)
 - la techno utilisée (requests + BS, pandas, pymongo)
 - le volume taille (en nb de lignes obtenues ou en taille de fichier)
 - les champs ("id, name, weight")
-
- Si fichiers séparés, faites un fichier unique (soit via un JOIN, soit concaténation de tables)
 - Pour le scraping : mettez dans le rapport un screen de la page à scraper. Expliquez un peu les packages BeautifulSoup et requests
 - Idéalement, présentez le code pour le traitement des 5 sources. Si vous n'avez pas les 5 sources, donnez à l'oral des exemples de sources manquantes, et comment vous vous y seriez pris
 - Il faut absolument 5 sources différentes pour valider le bloc (même si la plupart des systèmes réels en ont bcp moins...) Si vous n'avez rien à mettre dans un système big data, importez-y des données tabulaires standard. L'important est de prouver votre maîtrise des technos associées
 - Ça n'est pas grave si vous requêtez une source qui n'est pas utilisée plus tard dans l'agrégation

C2 : requêter des données en SQL, depuis une DB ou un système de big data

- requêtes "de type SQL". il doit y en avoir plusieurs
- mention d'optimisations dans les critères : capable d'optimiser/conscience de cet enjeu (cout de calcul, requêtes n+1), éviter d'avoir du code vraiment sous-optimisé : pas besoin de faire du zèle au delà de ça.
- souligner les possibilités à certains moments d'optimiser (une grosse jointure intéressante à ne pas faire, une étape de calcul très longue)
- La documentation attendue des requêtes est très précise, mais beaucoup de requêtes sont très standard (SELECT ... WHERE ...).
- Faut-il forcément des optimisations SQL pour chaque projet ? (certaines requêtes sont des SELECT + JOIN assez simples)
- parler brièvement d'ORM si vous en utilisez

C3 : créer des règles d'agrégation de données issues de différentes sources

Vous devez prouver que vous savez harmoniser vos 5 sources.

- utilisez pandas pour l'agreg plutôt que SQL, et justifiez brièvement votre choix (idéalement faites un peu des 2, ce serait bien d'avoir au moins un JOIN)
 - ayez un fichier par source :
- ```
/agreg/
 /api.py
 /csv.py
 ...
```

## C4 : modéliser les données en DB, en respectant le RGPD

- vous devez mettre toutes vos données dans une DB (nous conseillons une DB relationnelle type postgres/mysql)
- la partie RGPD est nulle si vous n'avez pas de DP. Tâchez de la mentionner brièvement : "nous nous sommes assurés que...". Justifier le fait qu'il n'y a pas de données personnelles, voire dire ce qu'on ferait s'il y en avait
- les SGBDR sont interchangeables, justifiez votre choix très rapidement
- Que faire quant au RGPD s'il n'y a pas de données personnelles ?

## C5 : développer une API mettant à disposition le jeu de données

- API "sécurisée" : il faut authentifier les requêtes ici (comme sur E3)
- OpenAPI : savoir en parler, mais c'est fait tout seul par Swagger
- L'authentification peut être faite au niveau de l'API, ou juste de la DB, tant que l'accès aux données demande une auth

## E2 : veille, choix et paramétrage d'un service d'IA

Vous devez partir d'un besoin client (réel ou fictif), vous . Le cas pratique évalué a pour but l'installation et la configuration du service d'IA préconisé.

Le/la candidat(e) doit

- Présenter son dispositif de veille.
  - Présenter la problématique technique et fonctionnelle d'intelligence artificielle à adresser à partir de l'expression de besoin.
  - Présenter l'inventaire des outils et services d'intelligence artificielle accessibles et répondant au problème visé.
  - Présenter une préconisation d'un ou plusieurs services d'intelligence artificielle.
  - Présenter les étapes de configuration et d'installation d'un ou plusieurs services d'intelligence artificielle préconisé (s).
- automatiser la veille technique, justification pour le Big Data, possibilité d'outils comme Pandas pour l'agrégation.
  - pas de démonstration pour cette partie, 15' c'est très long, il faut être détaillés
  - possible d'avoir une thématique commune au projet E3-E4 pour ce livrable

## C6 : réaliser une veille technique et réglementaire

- bien dire "sur quoi" on fait une veille, parfois c'est pas clair
- Bcp d'infos pertinentes sur l'IA viennent d'agrégateurs de news (Hacker News, TheNextWeb, etc.), de Twitter, et de blogs de développeurs. Mettez en place une veille automatisée autour de ces sources (flux RSS, etc.)

- la veille automatisée peut utiliser des outils comme InoReader qui agrègent des RSS de différents sites
- pas besoin d'y passer des heures, montre` que vous êtes capables de mettre en place un dispositif pour rester au courant (les collections sur Twitter peut faire l'affaire aussi)

## C7 : identifier des services d'IA préexistants

RAS, bien donner les critères de choix, ce qu'on vire vs garde

## C8 : paramétrer un service d'IA

- attention, il y a aussi une mention monitoring : "Installation et configuration des outils de monitoring disponibles avec le service intégré"
- pas obligés de construire un système de gestion des droits
- auth pas obligatoire, il s'agit de configurer un service pré-existant (ex : pour utiliser une fonctionnalité avec GCP/Azure, gestion des droits d'accès aux buckets)

# E3 : intégrer, monitorer, déployer un service d'IA

**Mise en situation -> Projet 1 (CDOE)**

Pas obligé d'avoir un modèle local, ni d'entraînement

## C9 : développer une API exposant un modèle d'IA

- "les résultats des tests sont correctement interprétés" : RAS
- montrer qu'on connaît le cadre OWASP, pas de trucs de fous

## C10 : intégrer l'API d'un modèle ou d'un service d'IA dans une application

- attention, il faut proposer un renouvellement de l'authentification !

## C11 : déploiement/MLOps : Monitorer un modèle d'IA

- on attend un monitoring de modèle type mlflow

- si le modèle est un LLM, vous êtes libres de vos métriques : TTFT, latence, débit, métriques Red/Blue...
- attention à la confusion "monitoring de modèle" vs "monitoring d'appli" : la mention de kibana est une erreur ici
- L'environnement "bac à sable" peut se trouver en local
- Le mieux est un dashboard type Dash/Streamlit, mais au pire celui de mlflow est suffisant

## C12 : déploiement/MLOps : Programmer les tests automatisés d'un modèle d'IA

- ne parler des tests que dans C12
- intéressant de justifier le taux de couverture retenu

## C13 : déploiement/MLOps : Développer la CI d'un modèle d'IA

- pas besoin que l'entraînement du modèle déclenche un déploiement de celui-ci

# E4 : gestion de projet

- à faire avec E3 dans le projet chef d'oeuvre
- pas obligé d'utiliser des containers
- attention à la répétition dans les critères entre tests et CI
- C14/C15/C16 : "soft" skills, gestion de projets. Le projet technique est le prétexte pour développer ces compétences ici

## C14 : analyser le besoin d'un commanditaire d'une application intégrant un service d'IA

- plusieurs standards possibles d'accessibilité : WCAG, RG2AA + les recos de Valentin Hauy, etc.
- montrer que vous respectez la mise en forme de vos markdown de config, et justifiez que c'est le format standard des dev, donc pas de police, couleurs, contraste, etc. Bien le dire

## C15 : à partir de cette analyse de besoin, concevoir le cadre technique de l'application

- présentez un diagramme qui représente le trajet des données, sur n'importe quel diagramme d'étapes

## C16 : coordonner sa réalisation technique

- gestion de projet Agile obligatoire (cf. projet existant, expérience réelle en entreprise...)
- c'est l'organisation collective du projet et si possible l'agilité qui est intéressée ici
- si vous avez des éléments de votre alternance, c'est cool : extrayez le plus d'éléments et partez de ça
- pas intéressant de créer une orga fictive

## C17 : développer les composants techniques et les interfaces

- C'est le gros bloc, très fourre-tout : prévoir plus de temps pour celui-là
- La gestion des droits d'accès doit se faire aussi au niveau du site utilisateur de l'application
- idéalement les maquettes sont externes, mais ce n'est pas à vous de les faire, elles sont présumées. Quitte à ce que ce soit artificiel, mettre au moins un mini CDC, pour montrer que c'est un minimum maîtrisé ("je veux que mon appli respecte tel critère .." pour montrer que les critères sont validés)
- utilisez au moins une maquette et montrez que vous y adhérez dans votre app
- vous pouvez renvoyer aux parties précédentes pour éviter les redites

## C18 : automatiser les phases de tests lors du versionnement

- Il y a des tests évoqués en C9, C10, C11, C12, C13 et C17. Cette partie concentre la partie CI/CD, on montre ici qu'on peut les automatiser

## C19 : créer un processus de livraison continue

- RAS, déploiement qui intègre les tests de C18

# E5 : monitoring et résolution de bug

Le but de cet ensemble est de construire une application fictive monitorée (C20), présentant au moins un bug (C21). Vous devrez ensuite résoudre ce bug, et documenter sa résolution.

E5 se fait a priori dans un projet à part, mais vous pouvez garder une thématique commune à E3/E4 pour ce livrable, car il y a peu d'intérêt à faire du monitoring/détection de bug dans un projet vide.

## C20 : monitorer une application d'IA

Vous devez ici proposer une documentation technique du monitoring. L'idée est de mettre en place le monitoring et le log d'une application, pour pouvoir détecter et remonter automatiquement les incidents :

- Vos outils (collecteurs, logs, agrégateurs, filtres, dashboard, etc.) doivent être installés et opérationnels en local. Vos alertes sont configurées et fonctionnelles, les seuils sont définis (pas besoin d'en avoir 50, juste quelques uns)
- Le top est de mettre en place une stack de monitoring type ELK. Il vous faut juste un/des collecteurs, un agrégateur (Prometheus a priori), du log, et un dashboard (Grafana a priori)
- Il vous faut du log intégré au code de votre app. Minimum : ``import logging``. Bonus : Prometheus Loki. Super-bonus : stack ELK avec les 3 containers
- Il vous faut des alertes qui se déclenchent en fonction de différents seuils
- Attention aux solutions de monitoring "clé en main" compliquées, faites au plus simple
- Docker Compose est idéal pour ce projet. Vous devriez avoir en gros : un fichier ``docker-compose.yml``, un fichier ``prometheus.yml``, et un fichier ``dashboard.json`` pour configurer Grafana.
- Tests, monitoring et CI/CD doivent être implémentés et automatisés

Dans la doc :

- Définissez des métriques de monitoring, des seuils d'alerte pour chacune, et le choix de vos alertes (email, SMS, Slack...)
- Justifier le choix de vos outils de consolidation/suivi des indicateurs, et les outils de log
- Documentez l'installation et configuration de ces outils et leurs dépendances
- La documentation respecte l'accessibilité blabla

- attention à la confusion possible entre monitoring du modèle (C11) et monitoring du système (C20) : la mention de "feedback loop" dans C20 est une erreur

- les logs doivent surveiller des métriques : justifiez pourquoi on conserve/monitore tels ou tels logs

- montrez votre maîtrise du principe plutôt que des technos spécifiques

- intéressant de collecter du feedback utilisateur qui déclenche/remette en cause la pertinence du modèle, surtout si vous expliquez votre démarche

- pas d'obligation de mise en place de la réinjection

## C21 : résoudre un incident technique et documenter la solution

On teste ici votre mise en place du monitoring applicatif via la résolution d'un bug. Partez d'une appli existante et **introduisez-y une erreur, que vous corrigerez ensuite**.

- Vous devez avoir un bug détectable par un outil automatisé (par ex. des tests, un `assert()`...) ou un log, et que vous pouvez corriger avec du code.



- Vous devrez reproduire le bug, l'analyser, coder un fix, tester votre solution, documenter le problème, et déployer la solution.
- Idéalement vous avez à la fin un test qui passe pour prouver la résolution
- Votre explication est orale, mais vous devez montrer le tests, la branche de fix, le merge.
- Vous devez montrer le déploiement de votre solution via une CI (on conseille d'utiliser GitHub Actions pour la CI)
- Vous n'avez pas besoin en revanche de coder le fix en live, juste de pouvoir en parler
- Cette partie n'est pas très lourde, vous pouvez la faire à la fin
- Vous pouvez utiliser un LLM pour des idées de bug sur votre code